

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("/content/Marketing_and_Sales_dataset.csv")
```

```
df.head()
```

	TV	Radio	Social Media	Influencer	Sales
0	Low	3.518070	2.293790	Micro	55.261284
1	Low	7.756876	2.572287	Mega	67.574904
2	High	20.348988	1.227180	Micro	272.250108
3	Medium	20.108487	2.728374	Mega	195.102176
4	High	31.653200	7.776978	Nano	273.960377

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.isnull().sum()
```

	0
TV	0
Radio	0
Social Media	0
Influencer	0
Sales	0

dtype: int64

Checking for multicollinearity among independent variables (TV, Radio, Social Media) using correlation matrices and VIF.

Mapping TV column to convert the strings to numeric values inorder to calculate coefficient matrix and VIF

```
mapping = {
    'Low': 1,
    'Medium': 2,
    'High': 3
}

df['TV'] = df['TV'].map(mapping)
```

```
X = df[['TV', 'Radio', 'Social Media']]
corr_matrix = X.corr()
print(corr_matrix)
```

	TV	Radio	Social Media
TV	1.000000	0.803377	0.511758

Radio	0.803377	1.000000	0.629941
Social Media	0.511758	0.629941	1.000000

Interpreting coefficient matrix:

TV has strong correlation with Radio but moderate correlation with Social Media.

Radio has strong correlation with TV but moderate correlation with Social Media.

Social Media has moderate correlation with both TV and Radio.

Calculating VIF

```
from statsmodels.tools.tools import add_constant
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd

X = df[['TV', 'Radio', 'Social Media']]
X = add_constant(X)

vif = pd.DataFrame({
    "Variable": X.columns,
    "VIF": [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

print(vif)
```

	Variable	VIF
0	const	7.040920
1	TV	2.820621
2	Radio	3.451590
3	Social Media	1.658144

Interpretation of VIF:

TV with VIF of 2.821 has a moderate correlation

Radio with VIF of 3.452 also has a moderate correlation with other variables

Social Media with VIF of 1.658 has a very low correlation with other variables.

Start coding or [generate](#) with AI.

Building a multiple linear regression model using statsmodel

```
import statsmodels.api as sm
x = df[['TV', 'Radio', 'Social Media']]
y = df['Sales']
X = sm.add_constant(x)
model = sm.OLS(y, X).fit()
print(model.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Sales    R-squared:                0.904
Model:                  OLS      Adj. R-squared:           0.903
Method:                 Least Squares    F-statistic:            1781.
Date:                   Sun, 21 Jun 2026    Prob (F-statistic):      2.10e-288
Time:                   01:39:22    Log-Likelihood:         -2714.2
No. Observations:       572      AIC:                    5436.
Df Residuals:           568      BIC:                    5454.
=====
```

```

Df Model:          3
Covariance Type:  nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
const          -12.2889       3.099      -3.966      0.000     -18.375      -6.202
TV              77.3227       2.456     31.486      0.000      72.499     82.146
Radio           2.9792       0.234     12.746      0.000       2.520      3.438
Social Media    -0.1577       0.672     -0.235      0.815     -1.478      1.163
=====
Omnibus:                 60.101   Durbin-Watson:                 1.874
Prob(Omnibus):            0.000   Jarque-Bera (JB):            17.942
Skew:                     0.050   Prob(JB):                     0.000127
Kurtosis:                  2.138   Cond. No.                      61.5
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The general equation for multiple linear regression model is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

Y = predicted value of the dependent variable

β_0 = intercept (constant)

$\beta_1, \beta_2, \beta_3, \dots, \beta_n$ = regression coefficients

$X_1, X_2, X_3, \dots, X_n$ = independent (predictor) variable

Therefore. from the regression summary:

$$Y = -12.29 + 77.32(TV) + 2.98(Radio) + -0.16(Social Media)$$

Start coding or [generate](#) with AI.

• Evaluating model performance using Adjusted R-squared and individual predictor p-values

From the model summary, the adjusted R squared is 0.903 which means 90.3%. The high value adjusted R squared value indicated that the model is of a better fitting.

```
print(model.pvalues.round(3))
```

```

const          0.000
TV              0.000
Radio           0.000
Social Media    0.815
dtype: float64

```

TV and Radio have pvalues less than 0.05, which indicates that they are statistically significant in predicting sales in the presence of other variables. Social Media on the otherhand has pvalue of 0.815 which is greater than 0.05 which implies that social media does not contribute significantly to the model.

Start coding or [generate](#) with AI.

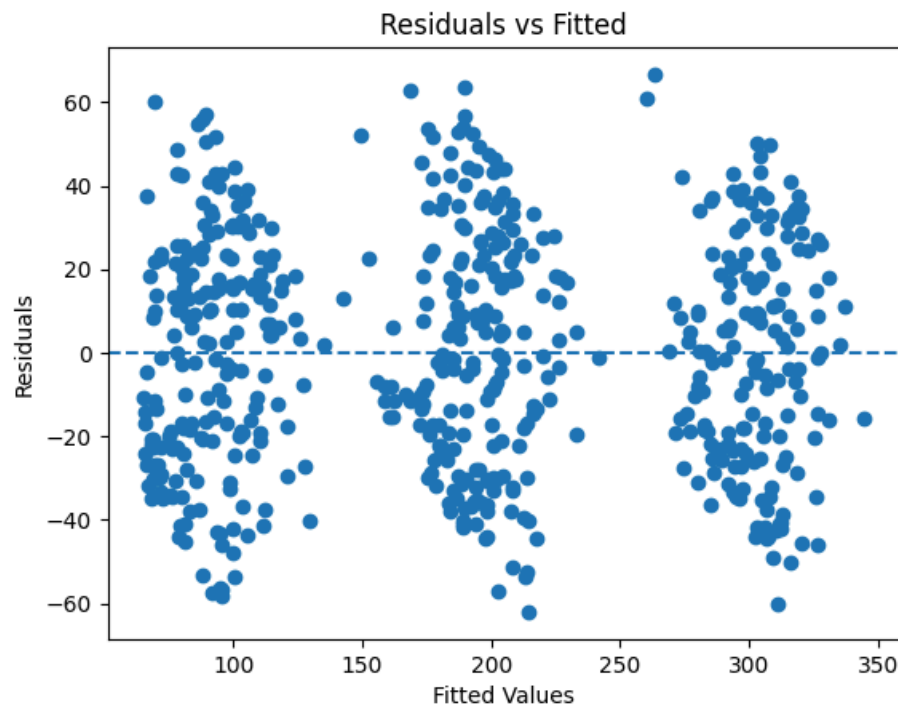
```
##checking the linearity
```

```

residuals = model.resid
fitted = model.fittedvalues

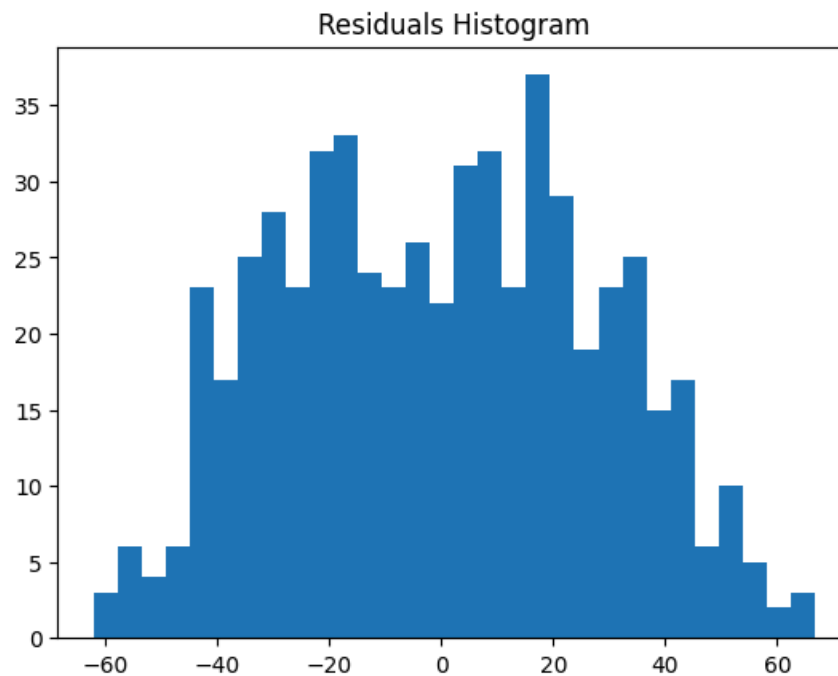
```

```
plt.scatter(fitted, residuals)
plt.axhline(y=0, linestyle='--')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted')
plt.show()
```



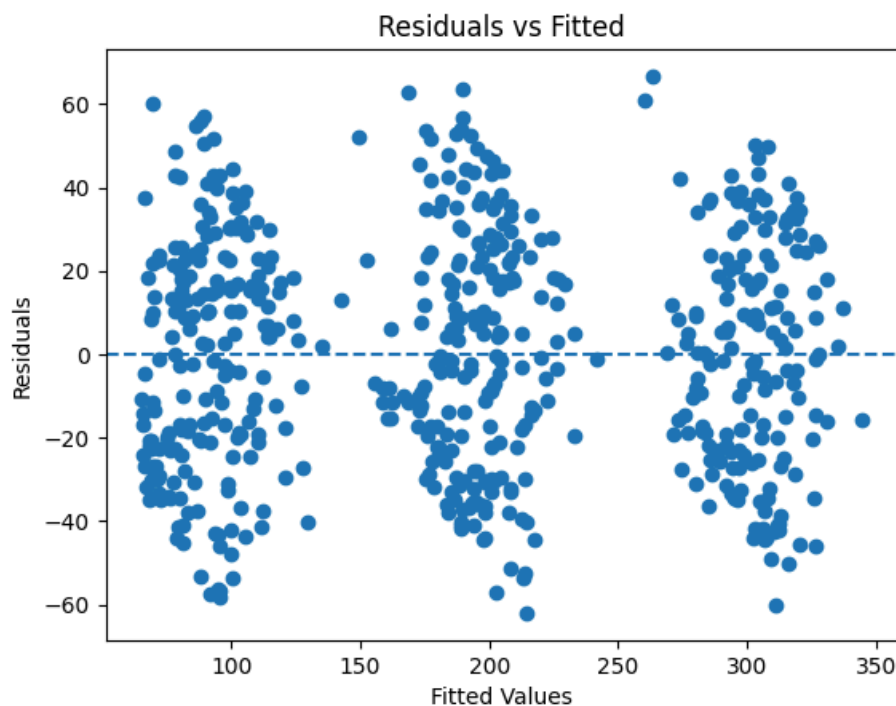
```
##checking the normality of the distribution
```

```
plt.hist(residuals, bins=30)
plt.title("Residuals Histogram")
plt.show()
```



```
##checking Homoscedasticity of residuals
```

```
plt.scatter(fitted, residuals)
plt.axhline(y=0, linestyle='--')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted')
plt.show()
```



The distribution is normal but not linear and no Homoscedasticity

```
tv_coef = model.params['TV']
print(tv_coef)
```

```
77.32268675374615
```

If other predicting variables of the model are kept constant, one unit increase in TV advertisement spending will bring about 77.32 units increase in sales.

```
Radio_coef = model.params['Radio']
print(Radio_coef)
```

```
2.979202236871191
```

If other predicting variables of the model are kept constant, one unit increase in Radio advertisement spending will bring about 2.97 units increase in sales.

```
Social_media_coef = model.params['Social Media']
print(Social_media_coef)
```

```
-0.15768873911379844
```

If other predicting variables of the model are kept constant, one unit increase in Social media advertisement spending will bring about 0.16 units decrease in sales.

Start coding or [generate](#) with AI.

The Business manager should invest more on TV advertisement since a unit increase in the TV advertisement cost leads 77.3 increase in sales. More of Radio advertisement should also be considered but social media advert should be dropped entirely or closer observation should be made in order to see a better way to go for the social media advertisement.

Start coding or [generate](#) with AI.